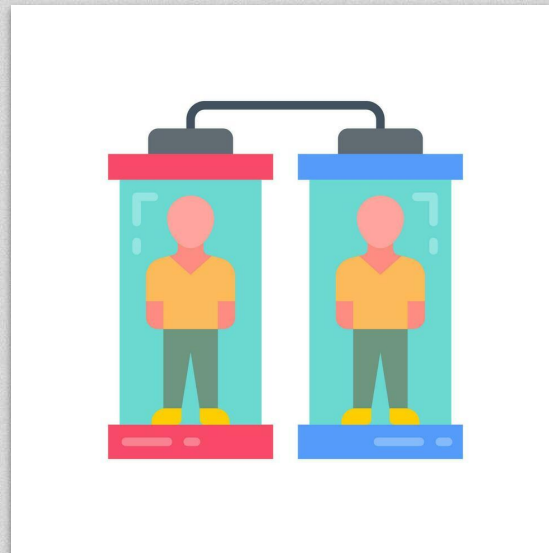


Prototyp



Bartłomiej Handziak



Cel wzorca projektowego

- ◇ Kreacyjny wzorzec projektowy
- ◇ **Pozwala tworzyć kopie istniejących obiektów** bez powiązania kodu z konkretnymi klasami tych obiektów



Opis problemu

- ◇ **Potrzeba uzyskania dokładnej (płytkiej lub głębokiej) kopii danego obiektu**
- ◇ Skopiowanie wszystkich pól oryginału do pól kopii
- ◇ Problem - nie wszystkie pola są dostępne np. pola prywatne, brak wiedzy o wszystkich polach
- ◇ Problem - kod zależny od kopiowanego obiektu, nie jest uniwersalny



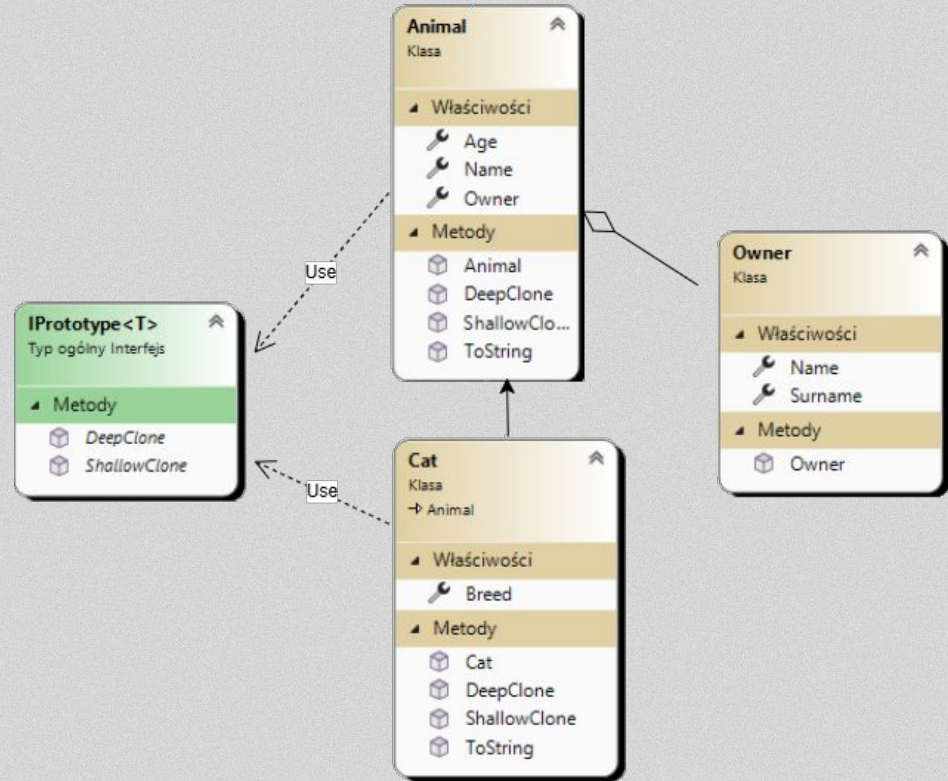
Rozwiązanie - implementacja prototypu

- ◇ Implementacja interfejsu IPrototype
- ◇ Zawiera metody ShallowClone() i DeepClone()
- ◇ Metoda tworzy obiekt swojej klasy i kopiuje do niej wartości jej pól.
- ◇ Można wtedy skopiować pola prywatne, ponieważ jest dostęp do nich wewnątrz klasy np. za pomocą wskaźnika *this*



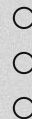
Schemat UML

Obiekt posiadający
funkcjonalność klonowania
nazywa się prototypem



Przykład w C#

```
1 Odwołanie  
public interface IPrototype<T>  
{  
    Odwołania: 4  
    T ShallowClone();  
    Odwołania: 3  
    T DeepClone();  
}
```



Definicja interfejsu generycznego

Przykład w C#

Odwołania: 4

```
public virtual Animal ShallowClone()
{
    return (Animal) this.MemberwiseClone();
}
```

Odwołania: 3

```
public virtual Animal DeepClone()
{
    Animal clone = (Animal) this.MemberwiseClone();
    clone.Owner = new Owner(Owner.Name, Owner.Surname);
    return clone;
}
```

Implementacja metod klonujących dla klasy Animal

